

VulnHGNN 2.0

SECURE C/C++ ANALYSIS & COMPILATION TOOLCHAIN

Project Technical Documentation & Target-State Specification

DOCUMENT PURPOSE : -

This document defines the finished state architecture, engineering boundaries, security model, interfaces, and acceptance criteria for VulnHGNN 2.0. It describes the evolution beyond the original VulnHGNN research prototype; it intentionally does not redocument capabilities that belong to Version 1 unless they are being replaced, hardened, or integrated into the new system.

Version 2.0 • Architecture baseline

Prepared as the engineering reference for implementation, validation, and deployment.

1. Executive Summary :-

→ VulnHGNN 2.0 is the next generation evolution of the existing VulnHGNN repository. The project does not replace the original research work. Instead, it keeps the useful research assets like program graph construction, learned representations, vulnerability detection, localization and repair concepts and restructures them into a dependable security pipeline that can operate as a developer tool.

→ **The defining change is architectural:** the system is no longer treated as a collection of ML experiments. It becomes a security toolchain with explicit contracts between source analysis, LLVM/IR analysis, graph construction, Agentic AI inference, localization, repair, verification, compilation, binary hardening and optional protection.

• What makes 2.0 different :-

- Detection is separated from localization, explanation, repair and verification.
- Clang/LLVM becomes the **compilation foundation** instead of attempting to build a new C/C++ compiler frontend Website.
- **Binary hardening and optional obfuscation** become measurable compiler/toolchain stages.
- The entire system is exposed through a stable **local CLI and machine-readable output**.
- **Agentic AI** is an orchestration layer over deterministic tools and evidence, not the security authority by itself.

VulnHGNN (Version 1) Research Prototype Graph Based Vulnerability Detection, Localization & Self Healing for C/C++	Major Differences	VulnHGNN 2.0 (Version 2) Local-first Security Toolchain Secure C/C++ Analysis & Compilation Toolchain
Overall Goal Research prototype for vulnerability detection, localization and self-healing using graph neural networks.	Purpose & Focus →	Purpose & Focus → Production-ready local security toolchain built on Clang/LLVM with explicit analysis, verification, compilation and binary security stages.
Architecture	System Architecture →	
Primary Use Research and experimentation.	Target Users →	Target Users → Developers, security researchers, students, and toolchain engineers (practical usage).
Interface Flask web UI and command-line pipeline for analysis.	User Interface →	User Interface → Stable local secure CLI with structured JSON output (developer toolchain).
ML Model VulnGAT (multi-label GNN) with sigmoid probabilities and per-class thresholds (e.g., 0.35 for CWE-476, 0.5 for others).	Model & Inference →	Model & Inference → Versioned inference engine with explicit graph contracts, reproducible evaluation and integration into a full toolchain.
Localization Identifies vulnerable regions (details not fully exposed in documentation).	Source Mapping →	Source Mapping → Precise mapping between model findings and source code, AST/IR and graph nodes.
Self Healing Rule-based remediation with feedback loop.	Repair & Validation →	Repair & Validation → AI-assisted repair generation with mandatory verification gate (compile, test, re-scan, behavior comparison).
Compilation clang with -O0 and debug information (used to generate LLVM IR).	Compilation Role →	Compilation Role → Full Clang/LLVM-based secure build pipeline (analysis + compilation + optimization).
Binary Security Not part of core system.	Binary Hardening →	Binary Hardening → Dedicated binary hardening/protection stage with measurable trade-offs (e.g., flags, obfuscation).
Verification Re-run analysis to check patched result.	Verification →	Verification → Comprehensive verification: compile, test, re-scan and compare behavior before accepting repairs.
AI Assistance Not a core component.	AI Role →	AI Role → Agentic AI as an evidence-grounded assistant for analysis, explanation, repair and tutoring (not the security authority).
Deployment Research repository (manual setup).	Distribution →	Distribution → Installable local developer toolchain (local-first, no cloud required for core workflow).

2. Version 2.0 Scope & Upgrade Boundary :-

→ The original VulnHGNN work established the research foundation. Version 2.0 focuses on everything required to turn that foundation into a reliable security product. Existing functionality is preserved when it is correct and useful; components that mix concerns, lack validation, duplicate experiments, or cannot support the new toolchain contracts are redesigned or replaced.

Area	Version 1 role	Version 2.0 transformation
IR / graph pipeline	Research data preparation	Canonical analysis pipeline with stable schemas and source/IR mapping
ML / GNN	Experimental vulnerability classifier	Versioned inference engine with explicit graph contracts and reproducible evaluation
Localization	Research output	Source ↔ AST ↔ LLVM IR ↔ graph node mapping
Healing	Prototype repair logic	Patch generation separated from verification and acceptance
Validation	Supporting experiments	Mandatory verification gate for accepted repairs
Compilation	Research preprocessing	Clang/LLVM based secure build pipeline

Binary security	Not the core system	Hardening/protection stage with measurable trade offs
Interface	Scripts / prototype app	Stable local secureCC CLI + JSON output
AI assistance	Feedback/experimental layer	Evidence grounded analyzer, explainer, repair and tutor orchestration
Deployment	Research repository	Installable local developer toolchain

3. Product Definition :-

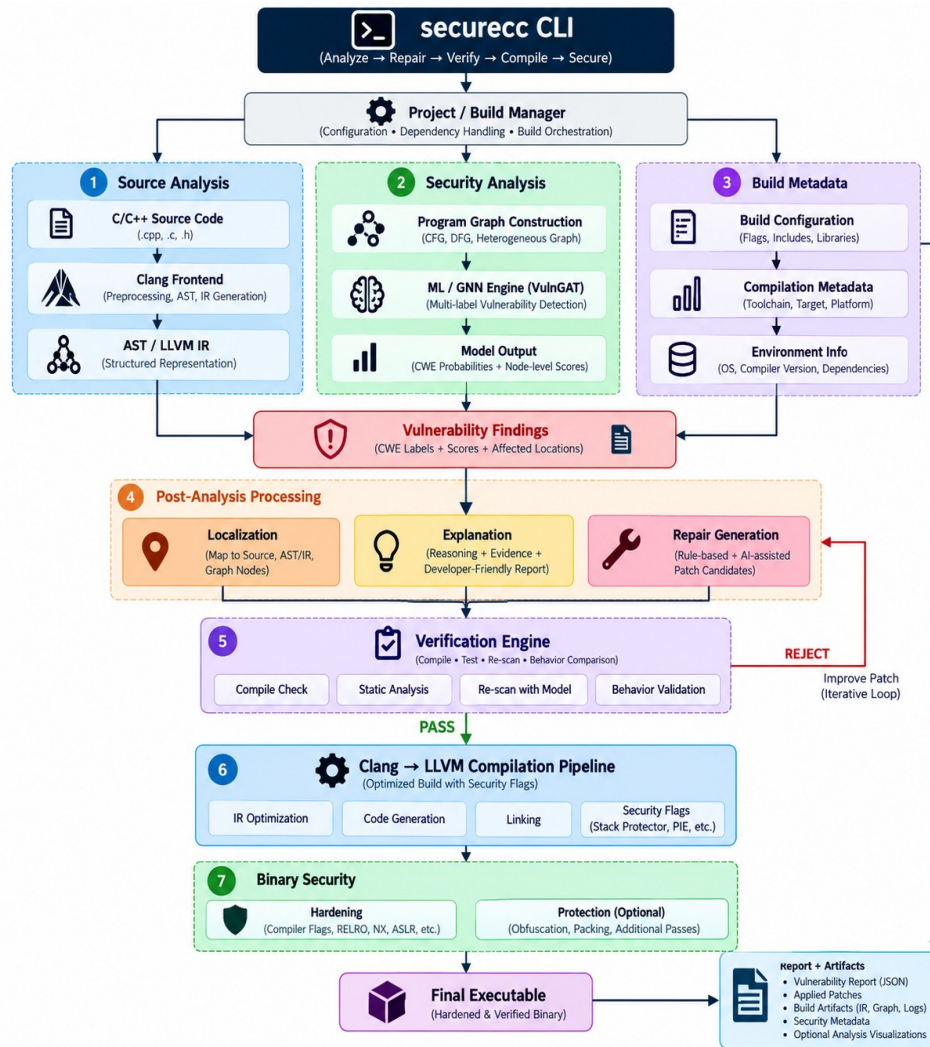
3.1 Product statement

VulnHGNN 2.0 is a local first C/C++ security compiler/toolchain that combines program analysis, heterogeneous program graphs, machine learning, repair verification and LLVM based compilation into a single developer workflow.

3.2 Primary users

- C/C++ developers who want security feedback during the build workflow.
- Security researchers studying graph based vulnerability detection.
- Students learning secure C/C++ programming through concrete findings.
- Toolchain engineers extending the LLVM and binary security stages.

4. Final System Architecture :-



4.1 Architectural layers

Layer	Responsibility	Primary output
Interface	CLI commands, configuration, machine readable output	Command result / JSON
Orchestration	Build and analysis lifecycle	Execution plan
Source analysis	Parsing, AST/source locations, compiler diagnostics	Source facts
Program graph	CFG, DFG, call and heterogeneous relations	Versioned graph
ML security	Detection, classification and node-level inference	Findings
Localization	Map findings to source and IR	Precise locations
Repair	Generate candidate patches	Patch candidates
Verification	Compile, test, re-scan and compare behavior	Verification decision
Compiler	Clang/LLVM compilation and optimization	Object/executable
Binary security	Hardening and optional protection	Protected binary
Reporting	Human and machine-readable evidence	Report/artifacts

5. VulnHGNN 2.0 Security Intelligence Engine :-

5.1 Improved Graph representation

→ The 2.0 engine treats program structure as a heterogeneous graph rather than a flat feature vector. Graph construction is an explicit subsystem and is validated independently of the model.

Entity / relation	Role in 2.0
Instruction / operation	Fine grained computational and memory behavior
Basic block	Control flow and local execution context
Function	Interprocedural context and call relationships
Variable / value	Data flow and def use relationships
CFG edge	Execution ordering / branching
DFG / def use edge	Value propagation
Call edge	Interprocedural context
Containment edge	Program hierarchy

5.2 AI/ML Model responsibilities

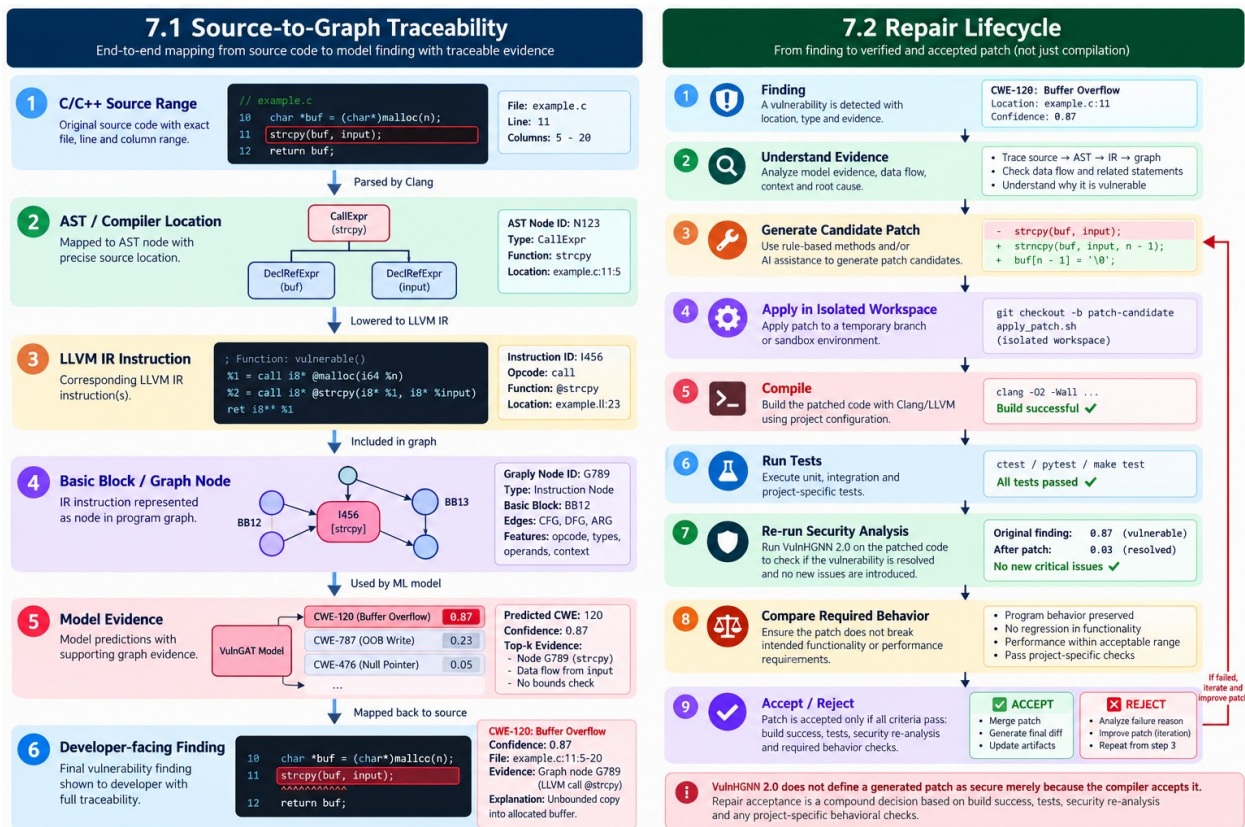
- Vulnerability detection at the appropriate graph/program granularity.
- CWE aware classification where supported by validated training data.
- Node or region localization where labels and mappings are available.
- Confidence estimation that is calibrated and not presented as proof of safety.

5.3 Research discipline

- Dataset splits are performed to prevent source family leakage.
- Training, validation and test data are versioned.
- Model checkpoints are tied to graph/feature/dataset versions.
- Per CWE metrics are reported rather than hiding weak classes behind one aggregate score.
- A simple baseline remains available to distinguish genuine graph learning gains from feature engineering gains.

6. Localization, Explanation & Repair :-

- Source to graph traceability & Repair life cycle



7. Binary Hardening & Protection :-

The binary security subsystem is designed to increase resistance to exploitation and reverse engineering while preserving correctness. It does not claim that a native binary can be made impossible to reverse engineer.

7.3 Required measurements

- Compiler/linker security:** Apply platform/profile specific security properties and protections.
- Metadata & memory safety:** Minimize unnecessary metadata and enable supported control-flow/memory protections.
- Binary validation:** Validate the final binary against the active security policy.

7.3 Required measurements

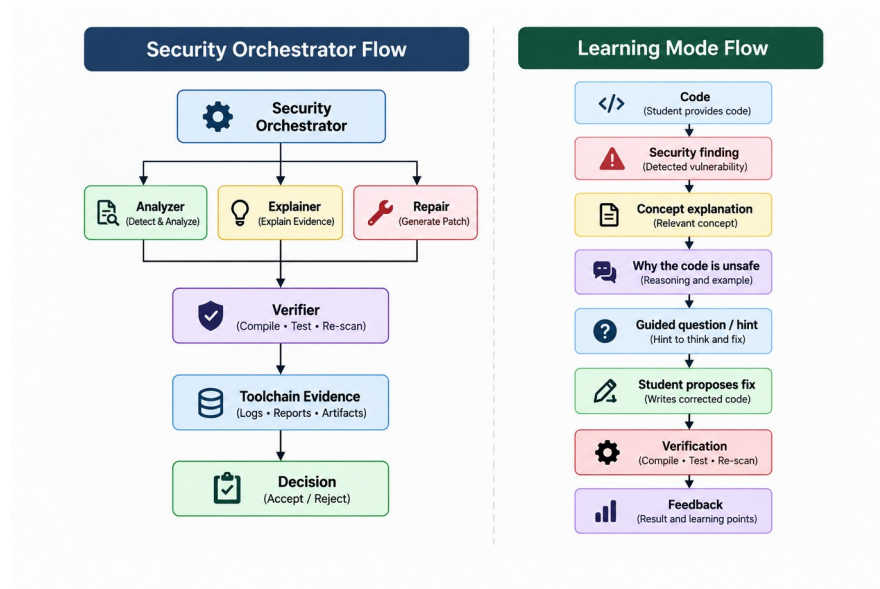
- Policy controlled:** Apply optional protection/obfuscation transformations as configured.
- Measured impact:** Benchmark every transformation against an unprotected baseline for security, performance, and compatibility.
- No absolute guarantees:** Prioritize correctness and express security gains as measured resistance/cost increases.

7.3 Required measurements

Metric	Purpose
Runtime overhead	Measure performance cost
Binary size	Measure deployment/storage impact
Startup time	Measure initialization cost
Memory use	Measure runtime resource impact
Compatibility	Detect platform/toolchain regressions
Security properties	Confirm requested hardening/protection is actually present
Transformation stability	Detect correctness failures

8. Agentic Security & Learning Layer :-

→ The agentic layer is deliberately downstream of deterministic evidence producing tools. Agents may reason over findings, explain concepts, propose repairs and guide learners, but security acceptance remains grounded in compiler, test and analysis results.



9. SecureCC Local CLI :-

→ The CLI is the primary product interface of VulnHGNN 2.0. The internal implementation can evolve without forcing developers to understand the research pipeline.

9.1 Machine readable interface

```
securecc scan project/ --format json
securecc build app.cpp --secure --report build/report.json
securecc verify build/app
```

9.2 Local first behavior

Command	Purpose
secureCC scan <file/project>	Run local security analysis
secureCC build <target>	Compile using the configured toolchain
secureCC build <target> --secure	Analyze, verify policy, compile and apply configured hardening
secureCC explain <finding>	Explain evidence and security implications
secureCC fix <target>	Generate and validate repair candidates
SecureCC verify <target>	Run verification checks
SecureCC harden <binary>	Apply/validate binary hardening
SecureCC protect <binary>	Apply configured protection transformations
secureCC learn <file>	Launch evidence based secure programming guidance

10. Security Model & Trust Boundaries :-

→ VulnHGNN 2.0 treats its own components as different trust domains. A model prediction is evidence, not authority; a generated patch is a proposal, not a trusted change; and a binary transformation is accepted only after verification.

Component	Trust role	Required control
Parser / compiler	Structured program facts	Compiler/toolchain validation
Graph builder	Program representation	Graph consistency tests
ML model	Probabilistic evidence	Calibration, thresholds, evaluation
Repair generator	Untrusted proposal	Isolation + compile + test + re-scan
AI agent	Reasoning assistant	Tool grounded evidence + bounded actions
Binary transformer	Security transformation	Correctness + benchmark + policy checks
CLI	User facing orchestrator	Input validation + safe process execution

11. Verification, Testing & Release Engineering :-

11.1 Release gate

- All critical tests pass.
- No known security regression is introduced.
- Model/data versions are recorded.
- Compiler/toolchain versions are recorded.
- Security profile is reproducible.
- Binary security checks pass where enabled.
- CLI output schemas remain compatible or are versioned.

11.2 Test layers

Layer	Examples
Unit	Graph utilities, parsers, feature encoders, policy logic
Integration	Source → IR → graph → inference → finding
Compiler	C/C++ build matrix, flags, LLVM pass behavior
Security regression	Known vulnerable and patched samples
Repair	Compile/test/rescan acceptance cases
Binary	Hardening/protection correctness
Performance	Runtime, memory, binary size, analysis latency
CLI	Commands, exit codes, JSON schema, error handling

VulnHGNN 2.0 Definition of Done :-

→ VulnHGNN 2.0 is considered complete when the following statements are demonstrably true through tests, benchmarks and reproducible demonstrations.

Area	Completion condition
Security engine	The validated ML/graph pipeline produces reproducible findings on the supported evaluation set.
Localization	A finding can be mapped to a developer meaningful source location with supporting evidence.
Repair	Supported repairs are generated as candidates and are never accepted without verification.
Verification	Compilation, tests and security reanalysis are integrated into the repair/build decision.
Compiler	C/C++ projects can be compiled through Clang/LLVM using the SecureCC workflow.
Hardening	Configured binary hardening is applied and its presence is validated.
Protection	Optional protection transformations are measurable, policy controlled and correctness tested.
CLI	A developer can use secureCC without interacting directly with internal Python research modules.
Offline core	Core analysis and compilation do not require cloud services.
Observability	Reports contain versions, policies, findings, verification results and artifact references.
Testing	Unit, integration, regression and performance tests protect the critical path.
Documentation	Installation, CLI usage, architecture, security boundaries and limitations are documented.